

L – Liskov Substitution (Sustitución de Liskov)

Una subclase debe poder usarse en lugar de su clase base sin romper nada. Si un método espera un `Animal`, cualquier `Perro` o `Gato` que herede de `Animal` debe comportarse de forma compatible: no puede exigir más precondiciones (“solo acepto animales ya alimentados”), ni devolver menos resultados o contratos más débiles que la clase base. Traducción brutal: heredar no es “agregar caprichos”, es respetar el contrato. Señales de violación típicas: métodos que lanzan “`UnsupportedOperationException`” en la subclase, “setters” que cambian reglas básicas (el clásico `Rectángulo/Cuadrado`), o subtipos que necesitan `if (instanceof...)` por todos lados. Solución: ajustar el modelo (usar composición en vez de herencia cuando el comportamiento diverge), mantener invariantes claros y documentar pre/postcondiciones que las subclases deben cumplir.

I – Interface Segregation (Segregación de Interfaces)

Es mejor tener varias interfaces pequeñas y enfocadas que una gorda que obligue a implementar métodos inútiles. Los clientes no deberían depender de lo que no usan. Si haces una interfaz `Notifier` con `sendEmail()`, `sendSMS()`, `sendPush()`, terminas con implementaciones tirando “no aplico”. Partilo en `EmailSender`, `SmsSender` y `PushSender`, y que cada clase implemente solo lo que necesita. Beneficios: menos código muerto, menos acoplamiento, tests más simples y cambios localizados. En Java moderno, las `default methods` pueden ayudar a evolucionar interfaces, pero no las uses para encubrir diseños inflados: si tenés muchos defaults vacíos, tu interfaz ya está pidiendo dividirse.